

Understanding Artifact Builds

Firefox for Desktop and Android supports a **fast build mode** called *artifact mode*. The resulting builds are called *artifact builds*. Artifact mode downloads pre-built C++ components rather than building them locally, trading bandwidth for time.

Artifact builds will be useful to many developers who are not working with compiled code (see “Restrictions” below). Artifacts are typically fetched from [mozilla-central](#).

To automatically download and use pre-built binary artifacts, add the following lines into your [mozconfig](#) file:

```
# Automatically download and use compiled C++ components:
ac_add_options --enable-artifact-builds

# Write build artifacts to:
mk_add_options MOZ_OBJDIR=./objdir-frontend
```

To automatically download and use the debug version of the pre-built binary artifact (currently supported for Linux, OSX and Windows artifacts), add `ac_add_options --enable-debug` to your mozconfig file (with artifact builds option already enabled):

```
# Enable debug versions of the pre-built binary artifacts:
ac_add_options --enable-debug

# Automatically download and use compiled C++ components:
ac_add_options --enable-artifact-builds

# Download debug info so that stack traces refers to file and columns
rather than library and Hex address
ac_add_options --enable-artifact-build-symbols

# Write build artifacts to:
mk_add_options MOZ_OBJDIR=./objdir-frontend-debug-artifact
```

Prerequisites

Artifact builds are supported for users of Mercurial and Git. Git artifact builds require a mozilla-central clone made with the help of [git-cinnabar](#). Please follow the instructions on the git-cinnabar project page to install git-cinnabar. Further information about using git-cinnabar to interact with Mozilla repositories can be found on [the project wiki](#).

Building

If you've added `--enable-artifact-builds` to your `mozconfig`, each time you run `mach build` and `mach build path/to/subdirectory` the build system will determine what the best pre-built binary artifacts available are, download them, and put them in place for you. The computations are cached, so the additional calculations should be very fast after the up-to-date artifacts are downloaded – just a second or two on modern hardware. Most Desktop developers should find that

```
./mach build  
./mach run
```

just works.

To only rebuild local changes (to avoid re-checking for pushes and/or unzipping the downloaded cached artifacts after local commits), you can use:

```
./mach build faster
```

which only “builds” local JS, CSS and packaged (e.g. images and other asset) files.

Most Firefox for Android developers should find that

```
./mach build  
./mach package  
./mach install
```

just works.

Pulling artifacts from a try build

To only accept artifacts from a specific revision (such as a try build), set `MOZ_ARTIFACT_REVISION` in your environment to the value of the revision that is at the head of the desired push. Note that this will override the default behavior of finding a recent candidate build with the required artifacts, and will cause builds to fail if the specified revision does not contain the required artifacts.

Pulling artifacts from local build / remote URL

If you need to do an artifact build against a local build or one hosted somewhere, you need to make use of respectively `MOZ_ARTIFACT_FILE` or `MOZ_ARTIFACT_URL`. In case of a local build, you will have to make sure you

- produce a package using `./mach package`
- point to it via `MOZ_ARTIFACT_FILE=path/to/firefox.tar.bz2` on your `./mach build` command line. The path needs to be absolute, and the package is under your object directory within `dist/`.

Using `MOZ_ARTIFACT_URL` will download the package at the given URL and then follow the same process as the local build case.

`MOZ_ARTIFACT_FILE` and `MOZ_ARTIFACT_URL` only provide the package, they do not provide sibling artifacts including the test artifacts, extra archives such as XPT data, etc. In general, prefer `MOZ_ARTIFACT_REVISION`, which will can provide these sibling artifacts.

Restrictions

Oh, so many. Artifact builds are rather delicate: any mismatch between your local source directory and the downloaded binary artifacts can result in difficult to diagnose incompatibilities, including unexplained crashes and catastrophic XPCOM initialization and registration failures. These are rare, but do happen.

Things that are supported

- Modifying JavaScript, (X)HTML, and CSS resources; and string properties and FTL files.
- Modifying Android Java code, resources, and strings.
- Running mochitests and xpcshell tests.
- Modifying `Scalars.yaml` to add Scalar Telemetry (since [Bug 1425909](#), except artifact builds on try).
- Modifying `Events.yaml` to add Event Telemetry (since [Bug 1448945](#), except artifact builds on try).

Essentially everything updated by `mach build faster` should work with artifact builds.

Things that are not supported

- Support for products other than Firefox for Desktop and Android are not supported and are unlikely to ever be supported. Other projects like Thunderbird may provide [their own support](#) for artifact builds.
- You cannot modify C, C++, or Rust source code anywhere in the tree. If it's compiled to machine code, it can't be changed.
- You cannot modify `histograms.json` to add Telemetry histogram definitions. (But see [Bug 1206117](#)).
- Modifying build system configuration and definitions does not work in all situations.

Things that are not yet supported

- Tests other than mochitests, xpcshell, and Marionette-based tests. There aren't inherent barriers here, but these are not known to work.
- Modifying WebIDL definitions, even ones implemented in JavaScript.

Troubleshooting

There are two parts to artifact mode: the `--disable-compile-environment` option, and the `mach artifact` command that implements the downloading and caching. Start by running

```
./mach artifact install --verbose
```

to see what the build system is trying to do. There is some support for querying and printing the cache; run `mach artifact` to see information about those commands.

Downloaded artifacts are stored in `$MOZBUILD_STATE_PATH/package-frontend`, which is almost always `~/.mozbuild/package-frontend`.

Discussion is best started on the [dev-builds mailing list](#). Questions are best raised in [#build](#) on [Matrix](#). Please file bugs in *Firefox Build System* :: *General*, blocking [Bug 901840](#)